

# 1 Module 1: Cheat Sheet for Data Preprocessing Tasks in Pandas & Plot Libraries

## 1.1 Data Preprocessing Tasks in Pandas

| Task                                 | Description                                       | Syntax                                                        | Example                                               |
|--------------------------------------|---------------------------------------------------|---------------------------------------------------------------|-------------------------------------------------------|
| <b>Load CSV Data</b>                 | Read data from a CSV file into a Pandas DataFrame | <code>pd.read_csv('filename.csv')</code>                      | <code>df_can = pd.read_csv('data.csv')</code>         |
| <b>Handling Missing Values</b>       | Drop rows with missing values                     | <code>df.dropna()</code>                                      | <code>df_can.dropna()</code>                          |
| <b>Handling Missing Values</b>       | Fill missing values with a specified value        | <code>df.fillna(value)</code>                                 | <code>df_can.fillna(0)</code>                         |
| <b>Removing Duplicates</b>           | Remove duplicate rows                             | <code>df.drop_duplicates()</code>                             | <code>df_can.drop_duplicates()</code>                 |
| <b>Renaming Columns</b>              | Rename one or more columns                        | <code>df.rename(columns={'old_name': 'new_name'})</code>      | <code>df_can.rename(columns={'Age': 'Years'})</code>  |
| <b>Selecting Columns</b>             | Select a single column                            | <code>df['column_name']</code> or <code>df.column_name</code> | <code>df_can['Age']</code> or <code>df_can.Age</code> |
| <b>Selecting Columns</b>             | Select multiple columns                           | <code>df[['col1', 'col2']]</code>                             | <code>df_can[['Name', 'Age']]</code>                  |
| <b>Filtering Rows</b>                | Filter rows based on a condition                  | <code>df[df['column'] &gt; value]</code>                      | <code>df_can[df_can['Age'] &gt; 30]</code>            |
| <b>Applying Functions to Columns</b> | Apply a function to transform values              | <code>df['column'].apply(function_name)</code>                | <code>df_can['Age'].apply(lambda x: x + 1)</code>     |

|                                 |                                            |                                                                        |                                                                     |
|---------------------------------|--------------------------------------------|------------------------------------------------------------------------|---------------------------------------------------------------------|
|                                 | in a column                                |                                                                        |                                                                     |
| <b>Creating New Columns</b>     | Create a new column using existing columns | <code>df['new_column'] = expression</code>                             | <code>df_can['Total'] = df_can['Quantity'] * df_can['Price']</code> |
| <b>Grouping and Aggregating</b> | Group rows and apply aggregate functions   | <code>df.groupby('column').agg({'col1': 'sum', 'col2': 'mean'})</code> | <code>df_can.groupby('Category').agg({'Total': 'mean'})</code>      |
| <b>Sorting Rows</b>             | Sort rows based on a column                | <code>df.sort_values('column', ascending=True/False)</code>            | <code>df_can.sort_values('Date', ascending=True)</code>             |
| <b>Displaying First n Rows</b>  | Show first n rows                          | <code>df.head(n)</code>                                                | <code>df_can.head(3)</code>                                         |
| <b>Displaying Last n Rows</b>   | Show last n rows                           | <code>df.tail(n)</code>                                                | <code>df_can.tail(3)</code>                                         |
| <b>Checking for Null Values</b> | Check for null values in the DataFrame     | <code>df.isnull()</code>                                               | <code>df_can.isnull()</code>                                        |
| <b>Selecting Rows by Index</b>  | Select rows based on integer index         | <code>df.iloc[index]</code>                                            | <code>df_can.iloc[3]</code>                                         |
| <b>Selecting Rows by Index</b>  | Select rows in a range                     | <code>df.iloc[start:end]</code>                                        | <code>df_can.iloc[2:5]</code>                                       |
| <b>Selecting Rows by Label</b>  | Select rows using labels/index names       | <code>df.loc[label]</code>                                             | <code>df_can.loc['Label']</code>                                    |
| <b>Selecting Rows by Label</b>  | Select rows within a label range           | <code>df.loc[start:end]</code>                                         | <code>df_can.loc['Age': 'Quantity']</code>                          |
| <b>Summary</b>                  | Generate descriptive statistics            | <code>df.describe()</code>                                             | <code>df_can.describe()</code>                                      |

|                   |                                     |  |  |
|-------------------|-------------------------------------|--|--|
| <b>Statistics</b> | ve statistics for numerical columns |  |  |
|-------------------|-------------------------------------|--|--|

## 1.2 Plot Libraries

| Library           | Main Purpose                            | Key Features                                       | Programming Language  | Level of Customization | Dashboard Capabilities                           | Types of Plots Possible                                                            |
|-------------------|-----------------------------------------|----------------------------------------------------|-----------------------|------------------------|--------------------------------------------------|------------------------------------------------------------------------------------|
| <b>Matplotlib</b> | General-purpose plotting                | Comprehensive plot types and customization options | Python                | High                   | Requires additional components and customization | Line plots, scatter plots, bar charts, histograms, pie charts, box plots, heatmaps |
| <b>Pandas</b>     | Data manipulation with plotting support | Easy plotting directly on Pandas DataFrames        | Python                | Medium                 | Can be combined with web frameworks              | Line plots, scatter plots, bar charts, histograms, pie charts, box plots           |
| <b>Seaborn</b>    | Statistical data visualization          | Stylish and specialized statistical plots          | Python                | Medium                 | Can be combined with other libraries             | Heatmaps, violin plots, scatter plots, bar plots, count plots                      |
| <b>Plotly</b>     | Interactive data visualization          | Interactive web-based visualizations               | Python, R, JavaScript | High                   | Dash framework supports interactive dashboards   | Line plots, scatter plots, bar charts, pie charts, 3D plots, choropleth maps       |
| <b>Folium</b>     | Geospatial data visualization           | Interactive and customizable maps                  | Python                | Medium                 | Can integrate with dashboards and frameworks     | Choropleth maps, point maps, heatmaps                                              |
| <b>PyWaffle</b>   | Waffle chart visualization              | Specialized waffle chart support                   | Python                | Low                    | Can be combined with other libraries             | Waffle charts, square pie charts, donut charts                                     |